UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Yuliana Almunawaroh
Kelas	SI5C
Judul project / topik API	Topik 27 – Penggalangan Dana: Donasi
Tautan repositori	https://github.com/lily2428240145/tugas1-restful-2428240145
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	18 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

RESTful API Penggalangan Dana: Donasi merupakan API yang digunakan untuk mengelola data donasi, mulai dari melihat seluruh data donasi, mencari donasi berdasarkan ID atau program, menambahkan donasi baru, memperbarui data, hingga menghapus donasi. API ini dikembangkan menggunakan Node.js dan Express.js dengan format pertukaran data JSON serta menggunakan penyimpanan sementara di memori.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/	Tidak	200	Menampilkan identitas API dan daftar endpoint
2	GET	/donations	Tidak	200	Mengambil seluruh data donasi
3	GET	/donations/:id	Tidak	201	Mengambil data donasi berdasarkan ID
4	GET	/donations?program=nilai	Tidak	200	Mengambil donasi berdasarkan program yang dipilih
5	POST	/donations	Tidak	201	Menambahkan data donasi baru
6	PUT	/donations/:id	Tidak	200	Memperbarui data donasi berdasarkan ID
7	DELETE	/donations/:id	Tidak	200	Menghapus data donasi berdasarkan ID

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Menyimpan alamat server API lokal

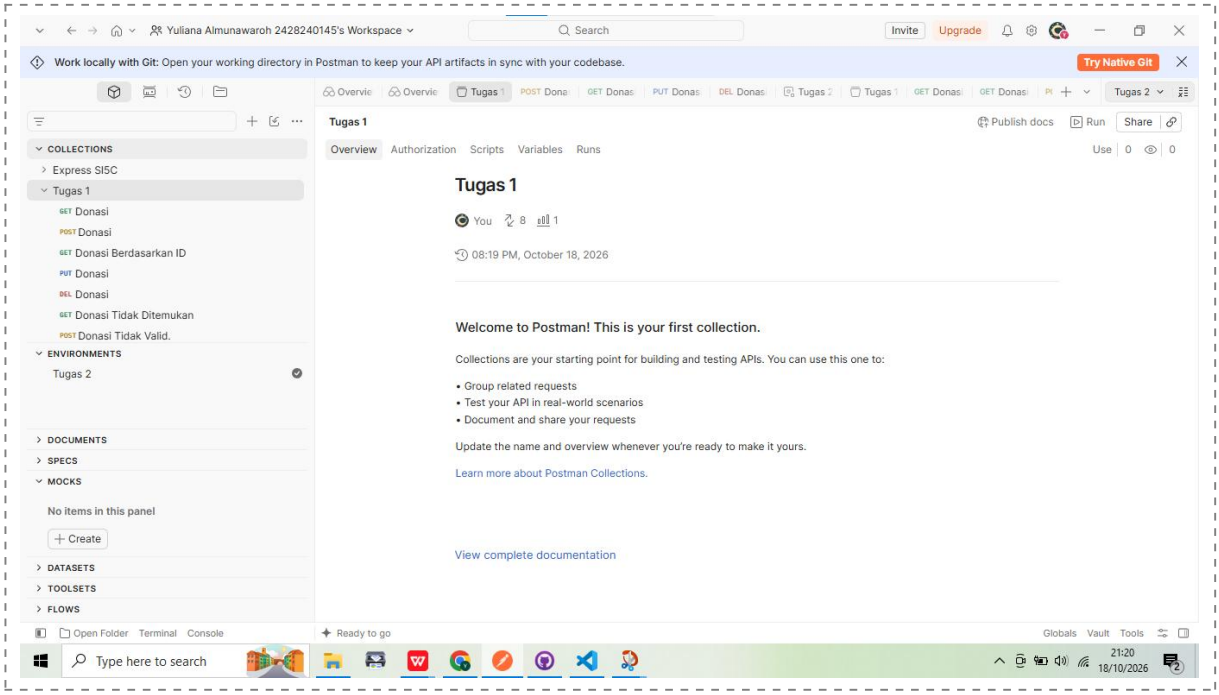
No	Nama variabel	Nilai (contoh)	Fungsi
2	donationId	Kosong, diisi script	Menyimpan ID donasi dari respons POST untuk digunakan pada GET, PUT, dan DELETE

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Seluruh request pengujian API Donasi dikelompokkan dalam Collection bernama Tugas 1. Request disusun berdasarkan fungsi CRUD, yaitu GET untuk mengambil data, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. Selain itu, terdapat request skenario gagal untuk menguji respons API ketika data tidak ditemukan atau data yang dikirim tidak valid. Pengelompokan ini memudahkan pengujian setiap fungsi API dan pelaksanaan seluruh automated test menggunakan Collection Runner.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Donasi	GET	{{base_url}}/donations	Tugas 1
2	POST Donasi	POST	{{base_url}}/donations	Tugas 1
3	GET Donasi Berdasarkan ID	GET	{{base_url}}/donations/{{donationId}}	Tugas 1

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
4	PUT Donasi	PUT	{{base_url}}/donations/{{donationId}}	Tugas 1
5	DEL Donasi	DELETE	{{base_url}}/donations/{{donationId}}	Tugas 1
6	GET Donasi Tidak Ditemukan	GET	{{base_url}}/donations/99999	Tugas 1
7	POST Donasi Tidak Valid	POST	{{base_url}}/donations	Tugas 1
8	POST Donasi Nominal Tidak Valid	POST	{{base_url}}/donations	Tugas 1
9	GET Informasi API	GET	{{base_url}}/	Tugas 1
10	GET Filter Donasi	GET	{{base_url}}/donations?program=Beasiswa%20Anak%20Desa	Tugas 1

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Donasi	200	Respons berupa array dan data memiliki field yang diperlukan
2	POST Donasi	201	Status sukses, data donasi benar, ID tersimpan ke environment
3	GET Donasi Berdasarkan ID	200	Respons berupa objek dan ID sesuai dengan ID yang tersimpan
4	PUT Donasi	200	Status sukses dan data donasi sudah diperbarui
5	DEL Donasi	200	Status sukses dan data bernilai null
6	GET Donasi Tidak Ditemukan	404	Pesan data tidak ditemukan dan struktur respons error sesuai
7	POST Donasi Tidak Valid	400	Pesan validasi gagal dan struktur respons error lengkap
8	POST Donasi Nominal Tidak Valid	400	Pesan nominal tidak valid dan struktur respons error lengkap

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
pm.test("Status code adalah 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response berupa array donasi", function () {
    const response = pm.response.json();

    pm.expect(response).to.be.an("array");
});

pm.test("Data donasi memiliki field yang diperlukan", function () {
    const response = pm.response.json();

    if (response.length > 0) {
        pm.expect(response[0]).to.have.property("id");
        pm.expect(response[0]).to.have.property("namaDonatur");
        pm.expect(response[0]).to.have.property("email");
        pm.expect(response[0]).to.have.property("nominal");
        pm.expect(response[0]).to.have.property("program");
        pm.expect(response[0]).to.have.property("anonim");
    }
});
```

Penjelasan:

Script GET Donasi memeriksa apakah server mengembalikan status HTTP 200 dan respons berbentuk array. Script juga memeriksa keberadaan field utama pada data donasi, yaitu id, namaDonatur, email, nominal, program, dan anonim. Dengan demikian, pengujian tidak hanya memastikan request berhasil, tetapi juga memeriksa struktur data yang dikembalikan API.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test("Status code adalah 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Response menunjukkan donasi berhasil dibuat", function () {
    const response = pm.response.json();

    pm.expect(response.status).to.eql("success");
    pm.expect(response.data).to.be.an("object");
});

pm.test("Data donasi berhasil disimpan", function () {
    const response = pm.response.json();
    const donasi = response.data;

    pm.expect(donasi).to.have.property("id");
    pm.expect(donasi.namaDonatur).to.eql("Yuliana Test");
    pm.expect(donasi.email).to.eql("yuliana.test@example.com");
    pm.expect(donasi.nominal).to.eql(150000);
    pm.expect(donasi.program).to.eql("Beasiswa Anak Desa");

    pm.environment.set("donationId", donasi.id);
});
```

Penjelasan:

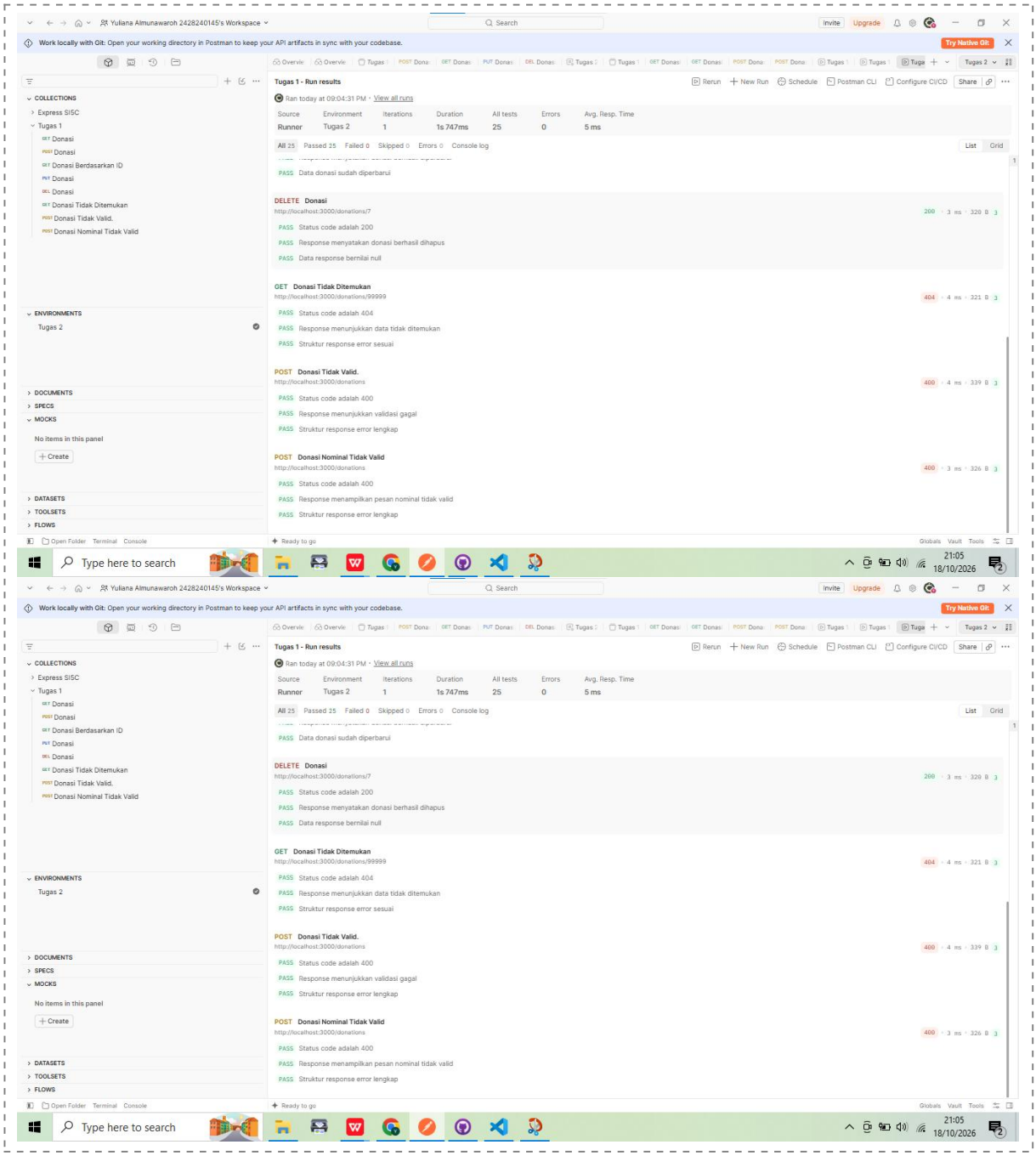
Script POST Donasi memeriksa apakah API mengembalikan status HTTP 201 ketika data baru berhasil ditambahkan. Pengujian juga memeriksa struktur dan isi data donasi yang dikembalikan oleh server. Setelah respons berhasil dan data dinyatakan sesuai, nilai ID donasi disimpan ke environment menggunakan variabel donationId. Variabel tersebut kemudian digunakan pada request GET berdasarkan ID, PUT, dan DELETE sehingga pengujian dapat dilakukan secara berurutan.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Pengujian RESTful API Penggalangan Dana: Donasi dilakukan menggunakan fitur Collection Runner pada Postman dengan Environment Tugas 2. Berdasarkan hasil pengujian, sebanyak 25 test berhasil dijalankan dengan status lulus, tanpa test gagal dan tanpa error. Sebelum pengujian dilakukan, alamat request diperiksa agar menggunakan base URL yang benar, yaitu http://localhost:3000. Pengujian otomatis membantu memastikan bahwa endpoint memberikan status HTTP dan respons JSON sesuai dengan skenario yang telah ditentukan.

4.2 Rekap angka hasil

Nama Collection	Tugas 1
Environment yang dipakai	Tugas 2
Jumlah request dijalankan	
Jumlah test lulus	25
Jumlah test gagal	0
Tanggal dan jam pengujian	18 Oktober 2026

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Request yang menggunakan URL lama	Request gagal terhubung atau mendapatkan respons yang tidak sesuai	URL menggunakan port 4000, sedangkan server berjalan pada port 3000	Mengubah URL agar menggunakan {{base_url}} dengan nilai http://localhost:3000, lalu menjalankan ulang Collection Runner

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

Tidak dikerjakan (pengujian Newman bersifat opsional)

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Pengujian menggunakan Newman belum dilakukan. Pengujian otomatis RESTful API Penggalangan Dana: Donasi telah dilakukan menggunakan fitur Collection Runner pada Postman. Hasil pengujian tersebut digunakan untuk memeriksa status respons HTTP dan kesesuaian struktur serta isi respons JSON pada setiap request yang diuji.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/lily2428240145/tugas1-restful-2428240145
Lokasi berkas Collection (.json)	Tugas 1
Lokasi berkas Environment (.json)	Tugas 2
Berkas Environment tidak memuat kunci rahasia sungguhan	[√] Ya [ ] Tidak
Repositori dapat dibuka dosen	[√] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Menurut saya, assertion yang paling berguna adalah pemeriksaan status HTTP dan kesesuaian isi respons JSON. Pemeriksaan status HTTP membantu mengetahui apakah request berhasil atau menghasilkan kesalahan sesuai skenario, misalnya status 201 saat menambahkan donasi dan status 404 ketika data tidak ditemukan. Sementara itu, pemeriksaan isi JSON memastikan bahwa data donasi yang dikembalikan sesuai dengan data yang dikirim atau diperbarui. Dengan kedua pemeriksaan tersebut, kesalahan tidak hanya dilihat dari status respons, tetapi juga dari ketepatan data yang dihasilkan API.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Menurut saya, pengujian API secara manual mengharuskan setiap request dijalankan dan hasilnya diperiksa satu per satu. Sementara itu, Collection Runner dapat menjalankan beberapa request secara berurutan dan memeriksa hasilnya secara otomatis melalui test script, sehingga pengujian menjadi lebih praktis dan konsisten. Bagian yang paling sulit adalah mengatur variabel donationId agar ID dari hasil penambahan donasi dapat digunakan pada request GET berdasarkan ID, PUT, dan DELETE. Saya mengatasinya dengan menyimpan ID dari respons POST ke Environment, menggunakan variabel tersebut pada request berikutnya, serta memastikan urutan request dan alamat server sudah benar.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	√
2	Setiap endpoint API saya ada di Collection	√
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	√
4	Setiap request punya assertion status code dan struktur JSON	√
5	Nama setiap test berupa kalimat yang menyatakan harapan	√
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	√
7	Server dinyalakan ulang sebelum Runner dijalankan	√
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	√
9	Berkas Environment tidak memuat kunci rahasia sungguhan	√
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	√

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen